Building data course — Week 3 — fault detection in SQL and pandas

sparql-playground / lessons

Generated for print and offline reading

Week 3 — fault detection in SQL and pandas

Five daily lessons. Each day has two scripts that compute the same thing: `dayNN_pandas.py` reads the CSV export into pandas, `dayNN_sql.py` runs SQLite queries on the same rows (`uv run python lessons/course.py setup` loads them once). Run both with `uv run python lessons/course.py run N` and compare the printouts. Each lesson's two scripts are printed after it. Keep notes under the ignored `student-work/` folder.

Contents

- Day 11 — Indexes and EXPLAIN QUERY PLAN: why some lookups are instant and others scan everything

- Day 12 — Window functions: look at neighbours and rolling windows (LAG, LEAD, rolling mean, fan starts)
- Day 13 — Streaks (gaps and islands): how long has this been true?
- Day 14 — Write a fault rule (FC1, duct static too low) twice, and check it against Open-FDD
- Day 15 — Data quality before you trust a rule, and the Unit 1 challenge

---

Day 11 — Indexes and EXPLAIN QUERY PLAN: why some lookups are instant and others scan everything

Goal: read a database's query plan, see the difference between a seek and a scan, build an index and measure what it buys, then do the same three ways in pandas (mask, `.loc` on a sorted index, binary search).

Read

Every query has two parts: *what* you want and *how* the engine finds it. SQL only states the *what*. The *how* is the **query plan**, and SQLite shows it if you put `EXPLAIN QUERY PLAN` in front of the statement. Two words matter:

- **SEARCH**: the engine used an index (a B-tree) to jump to the matching rows. About O(log n)

to find the first match, plus O(k) to read the k rows that match.
- **SCAN**: the engine read every row and tested each one. O(n), every time you ask.

FDD means asking the same kinds of questions over and over: “everything in March”, “everything where static is high”. Whether those are seeks or scans decides if a rule over a few years of data takes milliseconds or minutes.

Today’s two questions, both on AHU_1’s 35,536 rows:

1. **A time range:** one week of rows. `timestamp_utc` is the table’s `PRIMARY KEY`, so SQLite already keeps a B-tree sorted on it. The plan says `SEARCH ahu_1 USING INDEX sqlite_autoindex_ahu_1_1 (timestamp_utc>? AND timestamp_utc<?)`. (The timestamps are text in one fixed format, so text order *is* time order and a range works on text.)
2. **A rare value:** rows where duct static is above 1.7 inH2O (32 rows). Nobody indexed `da_p_inwc`, so the plan says `SCAN ahu_1`: it reads all 35,536 rows to return 32.

Then we **build an index on that column**. We do it on a `TEMP` copy of the table, so the lessons database is not changed. The plan flips to `SEARCH ... USING INDEX copy_da_p (da_p_inwc>?)` and the query gets much faster.

pandas has no query planner. *You* choose the access path, and there are three:

Question	SQL (`day11_sql.py`)	pandas (`day11_pa`
look at every row	`SCAN` (no index)	boolean mask `h[h.ts >= a`
build a sorted structure once	`CREATE INDEX`	`set_index("timestamp_utc` O(n log n) once
jump to the range	`SEARCH ... USING INDEX`	`.loc[a:b]` on the sorted ind
do the jump by hand	(the B-tree does it)	`np.searchsorted(sorted_a` n) per bound

`np.searchsorted` is plain binary search: halve the sorted array until you find the spot. A B-tree is the same idea made friendly to disks (many keys per node, so fewer steps), and it stays sorted as rows are added, which an array does not.

Big-O intuition. A mask touches every row, so it costs about c·n per query. A sorted structure costs a **one-time build** of about n·log n, then about log n + k per query. If the build costs B, one scan costs S and one lookup costs L, the index pays for itself after about B / (S − L) queries. Both scripts print that number for the rare-value query. It is small here (roughly 8 to 10 queries) because the build is cheap. At 100 million rows the build costs more in absolute time, but the scan grows 1000× while the lookup barely moves, so the index wins by more and more. If you will only ask once, don't build an index: a scan is cheaper than build plus lookup.

An index is not free, and not always useful. It takes space, it must be updated on every insert (slowing writes), and it only helps queries that are *selective*. Asking for 40% of the table through an index means jumping around for 14,000 rows, and reading the table straight through is about as fast. Try it below.

Run

```
uv run python lessons/course.py run 11
```

That runs `day11_pandas.py`, then `day11_sql.py`. The row counts must match; the **timings differ** from run to run and machine to machine (the tests compare only the rows). Expected shape of the output (your milliseconds will vary):

```
One week of rows, by mask / .loc /
searchsorted: 2016 / 2016 / 2016
  mask 3.262 ms | .loc 0.037 ms | searchsorted
0.020 ms
  building the index once: 1.887 ms
Rows with duct static > 1.7 inH2O: 32 (lookup
finds 32)
  the sort pays for itself after about 8.0
queries
...
Plan, one week by timestamp_utc (the PRIMARY
KEY):
  SEARCH ahu_1 USING INDEX
sqlite_autoindex_ahu_1_1 (timestamp_utc>? AND
timestamp_utc<?)
Plan, da_p_inwc > 1.7 (no index on that
column):
  SCAN ahu_1
TEMP copy, before the index:
  SCAN ahu_1_copy
TEMP copy, after CREATE INDEX:
  SEARCH ahu_1_copy USING INDEX copy_da_p
(da_p_inwc>?)
```

The week is 2,016 rows (7 days × 288 five-minute rows) in every method. The pandas script ends with a scaling table: a mask over 10 million sorted floats takes milliseconds and grows with n, while `searchsorted` stays near two microseconds.

Try it

1. In both scripts raise the threshold to `1.2` inH2O (about 14,000 rows). Do the plans change? Is the indexed query still faster?
2. The TEMP copy has no primary key. Run `EXPLAIN QUERY PLAN` for the one-week range on

the *copy*, then `CREATE INDEX temp.j ON ahu_1_copy (timestamp_utc)` and ask again.
3. Why can't `set_index("timestamp_utc")` speed up "`da_p_inwc > 1.7`" in pandas?

Answers

1. The plan still says `SEARCH ... USING INDEX`, but the time is the same as the scan (about 13 ms either way on the author's machine). An index only wins when it lets the engine skip most of the table. When 40% of rows match, the cost is dominated by reading and returning those rows. Selectivity decides, not the plan wording.
2. Before: `SCAN ahu_1_copy` (a copy made with `CREATE TABLE ... AS SELECT` has no primary key). After: `SEARCH ahu_1_copy USING INDEX j (timestamp_utc>? AND timestamp_utc<?)`. The original table gets this for free from its `PRIMARY KEY`.
3. An index on the timestamp is sorted by timestamp, so it only answers questions about timestamps. Another filter needs another sort order. That is why the script builds `np.argsort(da_p_inwc)` (row numbers in value order) for the rare-value query: it is exactly what a second database index is.

Why it matters

Real FDD queries are almost always "this equipment, this time range, this point". The database must jump straight there, or a rule scales with the *whole history* instead of the slice it needs.

Production historians use a composite index on (series, time); Day 14 builds one on its results (`fc1_eq_ts`) for exactly this reason. Day 12 uses windows to look at neighbouring rows, and Day 13 turns that into streaks.

day11_pandas.py

```python
"""Day 11, pandas: three ways to find rows, and what an index buys.

    uv run python lessons/unit1-sqlite/day11_pandas.py

Two questions on AHU_1's history: "all rows in one week" (a time range) and "all rows where duct
static is above 1.7 inH2O" (a rare value). Each is answered by a boolean mask (look at every row),
by an index (set_index + sort_index + .loc), and by binary search
(np.searchsorted), which is the
same idea a database index uses.
Compare the printout with day11_sql.py: the row counts should match; the timings will not.
"""

import time
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

RANGE_START = "2026-05-01 00:00:00+00:00"  # the week we want: [start, end)
RANGE_END = "2026-05-08 00:00:00+00:00"
RANGE_LAST = (
    "2026-05-07 23:55:00+00:00"  # .loc slices include their end label, so name the last row
)
HIGH_INWC = 1.7  # a rare duct static pressure (inH2O)
```

```
def best_ms(fn, repeat=7):
    """Best of `repeat` runs, in milliseconds
         (the minimum is the least noisy)."""
    times = []
    for _ in range(repeat):
        t0 = time.perf_counter()
        fn()
        times.append((time.perf_counter() - t0)
         * 1000)
    return min(times)


# 1. Load: one AHU, three columns. The CSV is
         already in time order.
COLUMNS = ["timestamp_utc", "da_p_inwc",
         "discharge_air_temp_f"]
h = pd.read_csv(DATA / "AHU_1" /
         "history_wide.csv", usecols=COLUMNS)
h = h.sort_values("timestamp_utc",
         ignore_index=True)
print(f"AHU_1: {len(h):,} rows")

# 2. The time range three ways. All three must
         return the same 2,016 rows (7 days x
         288).
times = h["timestamp_utc"].to_numpy()  #
         already sorted
indexed =
         h.set_index("timestamp_utc").sort_index()
         # built once: O(n log n)


def by_mask():  # O(n): compare every row
    return h[(h["timestamp_utc"] >=
         RANGE_START) & (h["timestamp_utc"] <
         RANGE_END)]


def by_loc():  # O(log n + k): the index finds
         both ends, then slices
    return indexed.loc[RANGE_START:RANGE_LAST]
```

```
def by_search():  # O(log n + k): two binary
        searches, then a slice
    lo = np.searchsorted(times, RANGE_START,
        side="left")
    hi = np.searchsorted(times, RANGE_END,
        side="left")
    return h.iloc[lo:hi]


def build_index():
    return
        h.set_index("timestamp_utc").sort_index()


week = by_mask()
counts = (len(week), len(by_loc()),
        len(by_search()))
print("\nOne week of rows, by mask / .loc /
        searchsorted:", " / ".join(map(str,
        counts)))
print(f"  all three agree on the row count:
        {len(set(counts)) == 1}")
t_mask, t_loc, t_search, t_build = (best_ms(f)
        for f in (by_mask, by_loc, by_search,
        build_index))
print(f"  mask {t_mask:.3f} ms | .loc
        {t_loc:.3f} ms | searchsorted
        {t_search:.3f} ms")
print(f"  building the index once:
        {t_build:.3f} ms")

# 3. A rare value: a mask scans every row; a
        sorted copy answers with one binary
        search.
values = h["da_p_inwc"].to_numpy()


def rare_mask():  # O(n), every time
    return h[h["da_p_inwc"] > HIGH_INWC]


def build_sort():  # the "index": row numbers
        in value order, O(n log n)
    return np.argsort(values, kind="stable")
```

```
order = build_sort()
sorted_vals = values[order]


def rare_lookup():  # O(log n + k)
    return
         h.iloc[order[np.searchsorted(sorted_vals,
         HIGH_INWC, side="right") :]]


high = rare_mask()
print(
    f"\nRows with duct static > {HIGH_INWC}
         inH2O: {len(high)} (lookup finds
         {len(rare_lookup())})"
)
t_rare_mask, t_rare_lookup, t_rare_build = (
    best_ms(f) for f in (rare_mask,
         rare_lookup, build_sort)
)
print(f"  mask {t_rare_mask:.3f} ms | sorted
         lookup {t_rare_lookup:.3f} ms")
print(f"  building the sort once:
         {t_rare_build:.3f} ms")

# 4. Break-even: how many queries until the
         one-time build cost is paid back?
saving = t_rare_mask - t_rare_lookup
if saving > 0:
    print(f"  the sort pays for itself after
         about {t_rare_build / saving:.1f}
         queries")
else:
    print("  at this size the mask is already
         so cheap that the sort never pays for
         itself")

# 5. Scaling: the mask grows with n, the binary
         search barely moves.
print("\nScaling (random sorted floats, one
         lookup):")
```

```
print(f"{'n':>12} {'mask ms':>10}
          {'searchsorted ms':>16}")
rng = np.random.default_rng(0)
for n in (100_000, 1_000_000, 10_000_000):
    vals = np.sort(rng.random(n))
    t_scan = best_ms(lambda v=vals: v > 0.999,
          3)
    t_seek = best_ms(lambda v=vals:
          np.searchsorted(v, 0.999), 3)
    print(f"{n:>12,} {t_scan:>10.3f}
          {t_seek:>16.5f}")

week_summary = pd.DataFrame({"rows":
          [len(week)], "avg_static_inwc":
          [week["da_p_inwc"].mean()]})
RESULTS = {
    "week_rows": week,
    "week_summary": week_summary,
    "high_static_rows": high,
}
```

day11_sql.py

```
"""Day 11, SQL: EXPLAIN QUERY PLAN, and what an
        index buys.

    uv run python lessons/course.py setup
        # once: loads the CSVs into
        var/lessons.sqlite
    uv run python lessons/unit1-
        sqlite/day11_sql.py

The same two questions as day11_pandas.py on
        AHU_1: a time range, and a rare duct
        static value.
SQLite tells us how it will run each query
        (EXPLAIN QUERY PLAN). The time range
        seeks into the
PRIMARY KEY; the value filter has no index, so
        it scans. We then index a TEMP copy of
        the table
(the lessons database is left untouched) and
        time the same query before and after.
Compare the printout with day11_pandas.py: the
        row counts should match; the timings
        will not.
"""

import os
import sqlite3
import time
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT /
        "var" / "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv
        run python lessons/course.py setup")
conn = sqlite3.connect(DB)

PARAMS = {
```

```
    "start": "2026-05-01 00:00:00+00:00",  #
        the week we want: [start, end)
    "end": "2026-05-08 00:00:00+00:00",
    "high": 1.7,  # a rare duct static pressure
        (inH2O)
}

# 2. The time range. timestamp_utc is the
        PRIMARY KEY, so SQLite keeps a B-tree
        on it.
WEEK_SQL = """
SELECT timestamp_utc, da_p_inwc,
        discharge_air_temp_f
FROM ahu_1
WHERE timestamp_utc >= :start AND timestamp_utc
        < :end
"""
WEEK_SUMMARY_SQL = """
SELECT count(*) AS rows, avg(da_p_inwc) AS
        avg_static_inwc
FROM ahu_1
WHERE timestamp_utc >= :start AND timestamp_utc
        < :end
"""

# 3. A rare value in a column nobody indexed.
HIGH_SQL = """
SELECT timestamp_utc, da_p_inwc,
        discharge_air_temp_f
FROM ahu_1
WHERE da_p_inwc > :high
"""
# The same filter on a TEMP copy, which is
        where we will build an index.
COPY_SQL = """
CREATE TEMP TABLE ahu_1_copy AS
SELECT timestamp_utc, da_p_inwc,
        discharge_air_temp_f FROM ahu_1
"""
HIGH_COPY_SQL = HIGH_SQL.replace("FROM ahu_1",
        "FROM ahu_1_copy")
INDEX_SQL = "CREATE INDEX temp.copy_da_p ON
        ahu_1_copy (da_p_inwc)"
```

```
def best_ms(fn, repeat=7):
    """Best of `repeat` runs, in milliseconds
         (the minimum is the least noisy)."""
    times = []
    for _ in range(repeat):
        t0 = time.perf_counter()
        fn()
        times.append((time.perf_counter() - t0)
         * 1000)
    return min(times)


def plan(sql, params=()):
    """EXPLAIN QUERY PLAN as text lines: SEARCH
         = index seek, SCAN = every row."""
    return [row[3] for row in
         conn.execute("EXPLAIN QUERY PLAN " +
         sql, params)]


print(f"AHU_1: {conn.execute('SELECT count(*)
         FROM ahu_1').fetchone()[0]:,} rows")

print("\nPlan, one week by timestamp_utc (the
         PRIMARY KEY):")
week_plan = plan(WEEK_SQL, PARAMS)
print("  " + "\n  ".join(week_plan))
week = pd.read_sql_query(WEEK_SQL, conn,
         params=PARAMS)
t_range = best_ms(lambda:
         conn.execute(WEEK_SQL,
         PARAMS).fetchall())
print(f"One week of rows: {len(week)}
         ({t_range:.3f} ms)")

print(f"\nPlan, da_p_inwc > {PARAMS['high']}
         (no index on that column):")
high_plan = plan(HIGH_SQL, PARAMS)
print("  " + "\n  ".join(high_plan))
high = pd.read_sql_query(HIGH_SQL, conn,
         params=PARAMS)
print(f"Rows with duct static >
         {PARAMS['high']} inH2O: {len(high)}")
```

```
# 4. Index the TEMP copy and run the same query
        again.
conn.execute(COPY_SQL)
t_before = best_ms(lambda:
        conn.execute(HIGH_COPY_SQL,
        PARAMS).fetchall())
plan_before = plan(HIGH_COPY_SQL, PARAMS)
t_build = best_ms(
    lambda: (conn.execute("DROP INDEX IF EXISTS
        temp.copy_da_p"),
        conn.execute(INDEX_SQL)), 5
)
t_after = best_ms(lambda:
        conn.execute(HIGH_COPY_SQL,
        PARAMS).fetchall())
plan_after = plan(HIGH_COPY_SQL, PARAMS)
high_indexed = pd.read_sql_query(HIGH_COPY_SQL,
        conn, params=PARAMS)

print("\nTEMP copy, before the index:")
print("  " + "\n  ".join(plan_before))
print("TEMP copy, after CREATE INDEX:")
print("  " + "\n  ".join(plan_after))
print(f"Same rows with the index:
        {len(high_indexed)}")
print(
    f"  scan {t_before:.3f} ms | indexed lookup
        {t_after:.3f} ms | build the index
        {t_build:.3f} ms"
)

# 5. Break-even: how many queries until the
        one-time build cost is paid back?
saving = t_before - t_after
if saving > 0:
    print(f"  the index pays for itself after
        about {t_build / saving:.1f} queries")
else:
    print("  at this size the scan is already
        so cheap that the index never pays for
        itself")

conn.execute("DROP INDEX temp.copy_da_p")
```

```
conn.execute("DROP TABLE temp.ahu_1_copy")  #
        nothing was written to the lessons
        database

week_summary =
        pd.read_sql_query(WEEK_SUMMARY_SQL,
        conn, params=PARAMS)
RESULTS = {
    "week_rows": week,
    "week_summary": week_summary,
    "high_static_rows": high,
    "plans": {"week": week_plan, "high":
        high_plan, "before": plan_before,
        "after": plan_after},
}
```

Day 12 — Window functions: look at neighbours and rolling windows (LAG, LEAD, rolling mean, fan starts)

Goal: compare each row with the rows around it, in SQL window functions and in pandas `shift` and `rolling`: previous and next readings, a 1-hour rolling mean, and the moments the supply fan starts and stops.

Read

An ordinary SQL query treats each row alone. FDD rules constantly need a row's **neighbours**: "did the status just change?", "what was the reading five minutes ago?", "what is the average over the last hour?". A **window function** computes a value for each row from a *window* of related rows, without collapsing them the way `GROUP BY` does. The output has as many rows as the input.

```
LAG(duct_static_pa)  OVER (PARTITION BY
        equipment_id ORDER BY timestamp_utc)
        -- the previous row's value
LEAD(duct_static_pa) OVER (PARTITION BY
        equipment_id ORDER BY timestamp_utc)
        -- the next row's value
AVG(duct_static_pa)  OVER (PARTITION BY
        equipment_id ORDER BY timestamp_utc
                        ROWS BETWEEN 11
        PRECEDING AND CURRENT ROW)          --
        this row and the 11 before it
```

`PARTITION BY equipment_id` restarts the window for each AHU, so AHU_2's first row never sees AHU_1's last row. `ORDER BY timestamp_utc` is what makes "previous" mean *earlier*. `ROWS BETWEEN 11 PRECEDING AND CURRENT ROW` is a 12-row **frame**: 12 five-minute rows = one hour. The first rows of each AHU have fewer than 11 rows behind them. SQL just averages what exists. pandas needs `min_periods=1` to do the same (the default would give NaN until it has all 12).

Idea	`day12_sql.py`	`day12_pandas.py`
previous / next row	`LAG(x) OVER w` / `LEAD(x) OVER w`	`groupby(eq)[x].shift(1)` / `.shift(-1)`
15 minutes ahead	`LEAD(x, 3) OVER w`	`.shift(-3)`
change since last row	`x - LAG(x) OVER w`	`x - x.shift(1)`
1-hour rolling mean	`AVG(x) OVER (... ROWS BETWEEN 11 PRECEDING AND CURRENT ROW)`	`.rolling(12, min_periods=1).mean()`
fan start	`CASE WHEN fan = 1 AND LAG(fan) OVER w = 0 THEN 1 ELSE 0 END`	`(fan == 1) & (fan_prev == 0)`
first row has no previous	`LAG` is `NULL`; the `CASE` falls to `ELSE 0`	`shift` gives `NaN`; the comparison is `False`

`groupby(eq)` plays the part of `PARTITION BY equipment_id` again. Both are one pass over rows in `(equipment, time)` order, so O(n) (plus the sort). The rolling mean can look like it costs 12 operations per row, but both engines slide the window, adding the new row and dropping the oldest, so it stays O(n).

Rows, not time. `ROWS BETWEEN 11 PRECEDING` and `rolling(12)` count rows. They mean "one hour" only because the data has exactly one row every five minutes. Day 15 checks that, and Day 18 handles data that has gaps.

Units. The CSV logs duct static in inches of water, so the scripts convert to pascals first (`da_p_inwc * 249.08891`), and Day 12's numbers are in Pa. SI is what the fault rules use from here on.

Run

```
uv run python lessons/course.py run 12
```

That runs `day12_pandas.py`, then `day12_sql.py`. Both end by printing the same thing:

```
Fan cycling per AHU (duct static in Pa):
equipment_id  fan_starts  fan_stops
max_static_1h_mean_pa               first_start
avg_static_15min_after_start_pa
       AHU_1         123        122
427.7 2026-03-16 09:00:00+00:00
294.68
       AHU_2         129        128
417.3 2026-03-16 09:00:00+00:00
330.76

First 6 fan events on AHU_1 (start=1 / stop=1):
equipment_id             timestamp_utc
fan_start  fan_stop
       AHU_1 2026-03-16 09:00:00+00:00
1         0
       AHU_1 2026-03-16 23:05:00+00:00
0         1
       AHU_1 2026-03-17 08:35:00+00:00
1         0
       ...
```

(Times are UTC.) The fan starts about once a day and runs about 14 hours on a normal day. The "around the first fan start" block at the end shows `static_prev_pa` lagging one row behind `duct_static_pa`, and the rolling mean climbing slowly (6.0 to 16.9 to 27.8 to 38.8 Pa): a mean over an hour reacts slowly by design. That is why rules use the instantaneous value for "right now" and the mean for "has this been going on".

Notice the data itself: static sits at 5.6 Pa, then 137.5 Pa for several rows in a row. The historian keeps the last value until a reading changes enough (change-of-value logging), so repeated values are normal. Day 15 turns that into a quality check.

Try it

1. Why do both AHUs have one more fan start than fan stop?
2. In both scripts change `min_periods=1` / the frame so that the first 11 rows of each AHU are `NULL`/`NaN` instead (pandas: delete `min_periods=1`; SQL: wrap the average in `CASE WHEN row_number() ... >= 12`). What do you get? Which is the "right" first-row behaviour for a rule that must see a full hour?
3. A step of more than 100 Pa in one row: how would you find those rows with the columns the script already makes?

Answers

1. The last fan run is still going when the data ends (2026-07-17 09:55 UTC), so it has a start and no stop. The first row of each AHU has no previous row, so it can be neither a start nor a stop.
2. The first 11 rows per AHU become `NaN` (pandas) or `NULL` (SQL) for `static_1h_mean_pa`. For a rule that needs a full hour of history, that is right: a mean of the 3 rows that exist is not an hour mean. For a plot, a partial mean is friendlier. Pick on purpose; the two tools default differently (SQL averages what exists, pandas returns `NaN`).
3. The script already makes `static_step_pa = duct_static_pa - static_prev_pa`. Filter `abs(static_step_pa) > 100`: `windows[windows.static_step_pa.abs() > 100]` in pandas, `WHERE abs(static_step_pa) > 100` in

SQL. That is 373 rows (186 on AHU_1, 187 on AHU_2). Many are the fan starting (about +290 Pa) or stopping (about -290 Pa), and the jump lands one row after the status flips. Compare them with `fan_start` and `fan_stop`.

Why it matters

Almost every rule is a window function in disguise. “Fan started less than 15 minutes ago” (suppress alarms while the system settles), “static has been above setpoint for the last hour” (rolling mean), “the status changed” (LAG). Day 13 builds on `LAG` to count how long something stays the same, and Day 14’s confirmation step uses the result.

day12_pandas.py

```
"""Day 12, pandas: look at neighbours and
        windows -- shift, rolling, and fan
        starts and stops.

    uv run python lessons/unit1-
        sqlite/day12_pandas.py

For each AHU, in time order: the previous and
        next duct static reading (shift), a 1-
        hour rolling
mean (12 rows of 5 minutes), and the rows where
        the supply fan starts or stops. Duct
        static is
converted to pascals (SI) first. Compare the
        printout with day12_sql.py: the
        numbers should match.
"""

from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

PA_PER_INH2O = 249.08891  # the historian
        logged inches of water; the math is SI
        (pascals)
WINDOW_ROWS = 12  # 12 rows x 5 min = 1 hour
AHEAD_ROWS = 3  # 3 rows x 5 min = 15 minutes

# 1. Load: both AHUs, the columns we need, in
        (equipment, time) order.
COLUMNS = ["timestamp_utc",
        "supply_fan_status", "da_p_inwc"]
frames = []
for equip in ("AHU_1", "AHU_2"):
    df = pd.read_csv(DATA / equip /
        "history_wide.csv", usecols=COLUMNS)
    df.insert(0, "equipment_id", equip)
    frames.append(df)
```

```
h = pd.concat(frames, ignore_index=True)
h = h.sort_values(["equipment_id",
        "timestamp_utc"], ignore_index=True)

# 2. SI units
h["duct_static_pa"] = h["da_p_inwc"] *
        PA_PER_INH2O
eq = h["equipment_id"]
static = h.groupby(eq)["duct_static_pa"]  #
        groupby(eq) plays the part of
        PARTITION BY equipment_id
fan = h["supply_fan_status"]

# 3. LAG / LEAD are shift(+n) / shift(-n). An
        AHU's first row has no previous row:
        NaN (SQL: NULL).
h["static_prev_pa"] = static.shift(1)
h["static_next_pa"] = static.shift(-1)
h["static_ahead_pa"] = static.shift(-
        AHEAD_ROWS)
h["static_step_pa"] = h["duct_static_pa"] -
        h["static_prev_pa"]

# 4. A 1-hour rolling mean of the current row
        and the 11 before it. min_periods=1
        lets the first
# rows use what they have (SQL's ROWS BETWEEN
        11 PRECEDING does that without being
        asked).
h["static_1h_mean_pa"] =
        static.transform(lambda s:
        s.rolling(WINDOW_ROWS,
        min_periods=1).mean())

# 5. Fan starts and stops: the status changed
        from the previous row.
fan_prev = fan.groupby(eq).shift(1)
h["fan_start"] = ((fan == 1) & (fan_prev ==
        0)).astype(int)
h["fan_stop"] = ((fan == 0) & (fan_prev ==
        1)).astype(int)

WINDOW_COLUMNS = [
    "equipment_id",
```

```
    "timestamp_utc",
    "supply_fan_status",
    "duct_static_pa",
    "static_prev_pa",
    "static_next_pa",
    "static_ahead_pa",
    "static_step_pa",
    "static_1h_mean_pa",
    "fan_start",
    "fan_stop",
]
windows = h[WINDOW_COLUMNS]

# 6. One line per AHU: how often the fan
     cycles, and static 15 minutes after a
     start.
per_ahu = h.groupby("equipment_id").agg(
    fan_starts=("fan_start", "sum"),
    fan_stops=("fan_stop", "sum"),
    max_static_1h_mean_pa=("static_1h_mean_pa",
        "max"),
)
after_start = (
    h[h["fan_start"] == 1]
    .groupby("equipment_id")
    .agg(
        first_start=("timestamp_utc", "min"),
        avg_static_15min_after_start_pa=
         ("static_ahead_pa", "mean"),
    )
)
summary =
     per_ahu.join(after_start).reset_index()
changed = (h["fan_start"] == 1) |
     (h["fan_stop"] == 1)
events = h.loc[changed, ["equipment_id",
     "timestamp_utc", "fan_start",
     "fan_stop"]]

RESULTS = {"windows": windows, "fan_summary":
     summary, "fan_events": events}
```

```
print("Fan cycling per AHU (duct static in
        Pa):")
print(summary.round(2).to_string(index=False))
print("\nFirst 6 fan events on AHU_1 (start=1 /
        stop=1):")
print(events[events["equipment_id"] ==
        "AHU_1"].head(6).to_string(index=False))
sample = (
    windows[(windows["equipment_id"] ==
        "AHU_1") & (windows["fan_start"] ==
        1)].head(1).index[0]
)
print("\nAround the first AHU_1 fan start:")
cols = [
    "timestamp_utc",
    "supply_fan_status",
    "duct_static_pa",
    "static_prev_pa",
    "static_1h_mean_pa",
]
print(windows.loc[sample - 2 : sample + 3,
        cols].round(1).to_string(index=False))
```

day12_sql.py

```
"""Day 12, SQL: window functions -- LAG/LEAD, a
        1-hour rolling mean, and fan starts
        and stops.

    uv run python lessons/course.py setup
        # once: loads the CSVs into
        var/lessons.sqlite
    uv run python lessons/unit1-
        sqlite/day12_sql.py

The same computation as day12_pandas.py, done
        by SQLite on the same rows. SQLite
        does all the work;
pandas only turns the result rows into a table
        for printing.
Compare the printout with day12_pandas.py: the
        numbers should match.
"""

import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT /
        "var" / "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv
        run python lessons/course.py setup")
conn = sqlite3.connect(DB)

# the historian logged inches of water; the
        math is SI (pascals)
PARAMS = {"pa_per_inh2o": 249.08891}

# One CTE per step. The window is 12 rows x 5
        min = 1 hour; LEAD 3 looks 15 minutes
        ahead.
WINDOWS_SQL = """
```

```
CREATE TEMP TABLE windows AS
WITH h AS (                                        -
         - 1. load: both AHUs, the columns we
         need
  SELECT 'AHU_1' AS equipment_id,
         timestamp_utc, supply_fan_status,
         da_p_inwc FROM ahu_1
  UNION ALL
  SELECT 'AHU_2', timestamp_utc,
         supply_fan_status, da_p_inwc FROM
         ahu_2
), si AS (                                         -
         - 2. SI units
  SELECT equipment_id, timestamp_utc,
         supply_fan_status,
         da_p_inwc * :pa_per_inh2o AS
         duct_static_pa
  FROM h
), w AS (                                          -
         - 3-5. neighbours, rolling mean,
         previous fan status
  SELECT equipment_id, timestamp_utc,
         supply_fan_status, duct_static_pa,
         LAG(duct_static_pa)     OVER ord AS
         static_prev_pa,
         LEAD(duct_static_pa)    OVER ord AS
         static_next_pa,
         LEAD(duct_static_pa, 3) OVER ord AS
         static_ahead_pa,
         AVG(duct_static_pa) OVER (PARTITION BY
         equipment_id ORDER BY timestamp_utc
                                   ROWS BETWEEN
         11 PRECEDING AND CURRENT ROW) AS
         static_1h_mean_pa,
         LAG(supply_fan_status)  OVER ord AS
         fan_prev
  FROM si
  WINDOW ord AS (PARTITION BY equipment_id
         ORDER BY timestamp_utc)
)
SELECT equipment_id, timestamp_utc,
         supply_fan_status, duct_static_pa,
         static_prev_pa,
       static_next_pa, static_ahead_pa,
```

```
        duct_static_pa - static_prev_pa AS
          static_step_pa,
        static_1h_mean_pa,
        CASE WHEN supply_fan_status = 1 AND
          fan_prev = 0 THEN 1 ELSE 0 END AS
          fan_start,
        CASE WHEN supply_fan_status = 0 AND
          fan_prev = 1 THEN 1 ELSE 0 END AS
          fan_stop
FROM w
"""
conn.execute(WINDOWS_SQL, PARAMS)

SUMMARY_SQL = """
SELECT equipment_id,                          -
         - 6. one line per AHU
        sum(fan_start) AS fan_starts,
        sum(fan_stop)  AS fan_stops,
        max(static_1h_mean_pa) AS
          max_static_1h_mean_pa,
        min(CASE WHEN fan_start = 1 THEN
          timestamp_utc END) AS first_start,
        avg(CASE WHEN fan_start = 1 THEN
          static_ahead_pa END) AS
          avg_static_15min_after_start_pa
FROM windows
GROUP BY equipment_id
ORDER BY equipment_id
"""
EVENTS_SQL = """
SELECT equipment_id, timestamp_utc, fan_start,
         fan_stop
FROM windows
WHERE fan_start = 1 OR fan_stop = 1
ORDER BY equipment_id, timestamp_utc
"""

windows = pd.read_sql_query("SELECT * FROM
         windows ORDER BY equipment_id,
         timestamp_utc", conn)
summary = pd.read_sql_query(SUMMARY_SQL, conn)
events = pd.read_sql_query(EVENTS_SQL, conn)
```

```
RESULTS = {"windows": windows, "fan_summary":
          summary, "fan_events": events}

print("Fan cycling per AHU (duct static in
          Pa):")
print(summary.round(2).to_string(index=False))
print("\nFirst 6 fan events on AHU_1 (start=1 /
          stop=1):")
print(events[events["equipment_id"] ==
          "AHU_1"].head(6).to_string(index=False))
sample = (
    windows[(windows["equipment_id"] ==
          "AHU_1") & (windows["fan_start"] ==
          1)].head(1).index[0]
)
print("\nAround the first AHU_1 fan start:")
cols = [
    "timestamp_utc",
    "supply_fan_status",
    "duct_static_pa",
    "static_prev_pa",
    "static_1h_mean_pa",
]
print(windows.loc[sample - 2 : sample + 3,
          cols].round(1).to_string(index=False))
```

Day 13 — Streaks (gaps and islands): how long has this been true?

Goal: find runs of consecutive rows with the same value, using the three-step streak trick in SQL and its `cumsum` twin in pandas. Use it to find the longest continuous fan run and to count "true for N rows in a row", the confirmation step in every Open-FDD rule.

Read

A lot of FDD questions are about **duration**, not single readings: "the fan has been on for 30 minutes", "static has been low for an hour", "this sensor hasn't changed all night". A raw fault on one row is noise; a raw fault that holds for N rows is a finding. Rows in a table don't know their neighbours, so we need a way to say "these consecutive rows belong to one run". This is the classic *gaps and islands* problem.

The trick has three steps. Take the fan status as an example:

```
status        0 0 1 1 1 0 1 1
new_streak    1 0 1 0 0 1 1 0      1 where the
value differs from the previous row (the first
row is always 1)
streak_id     1 1 2 2 2 3 4 4      running
total of new_streak: every row of a run shares
one id
row in streak 1 2 1 2 3 1 1 2      counts 1, 2,
3 ... inside each run
```

Now a run is just a `GROUP BY streak_id`, and "true for N rows in a row" is `row in streak >= N`. Each step is cheap and the whole thing is one pass over the rows, O(n) after sorting.

Step	`day13_sql.py` (window functions)	`day13_pandas.py` (vect
mark changes	`CASE WHEN x = LAG(x) OVER w THEN 0 ELSE 1 END`	`x.ne(x.groupby(eq).shift(`
streak id	`SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY ts)`	`new_streak.groupby(eq).cu`
row in streak	`ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id ORDER BY ts)`	`x.groupby([eq, streak_id]).cumcount() +`
each run	`GROUP BY equipment_id, streak_id`	`groupby(["equipment_id", "run_id"]).agg(...)`

Step	`day13_sql.py` (window functions)	`day13_pandas.py` (vect
longest per AHU	rank with `ROW_NUMBER() OVER (PARTITION BY equipment_id ORDER BY rows DESC)`, keep 1	`sort_values(...).groupby(`

Two details. On an AHU's first row `LAG` is `NULL`, so `x = LAG(x)` is `NULL`, not false, and the `CASE` takes the `ELSE 1` branch: the first row starts a streak, as it should. And a *tie* for the longest run needs a rule, or two runs of equal length make the answer unstable; both scripts break ties by the earlier start.

A second route to "true for N rows in a row". Instead of numbering streaks, slide a window of N rows over the 0/1 raw flag and ask whether all N are 1: `SUM(raw) OVER (... ROWS BETWEEN 5 PRECEDING AND CURRENT ROW) = 6` in SQL, `raw.rolling(6).sum() == 6` in pandas (Day 12's windows). Both scripts compute both routes and print their counts side by side: they agree. The streak route needs no window size, and it gives you the streak id so you can count episodes and measure their length. The window route is shorter when you only need the flag.

Today's raw flag is "supply fan at 95% speed or more while running", confirmed after **6 rows** (30 minutes). Day 14 uses the same shape with a pressure rule.

Run

```
uv run python lessons/course.py run 13
```

That runs `day13_pandas.py`, then `day13_sql.py`. Both end by printing:

```
Fan runs per AHU (status = 1 streaks):
equipment_id  runs  avg_run_hours  longest_rows
longest_hours             longest_start
       AHU_1   123          12.80          1313
109.42 2026-07-06 09:45:00+00:00
       AHU_2   129          11.62          1313
109.42 2026-07-06 09:45:00+00:00

Full speed (>= 95 %) for 6 rows in a row, by
streak id and by rolling sum:
equipment_id  raw_rows  confirmed_rows_streak
confirmed_rows_window  episodes
       AHU_1      1349                   1153
1153        31
       AHU_2       222                    192
192         6
```

The fans normally run about 14 hours a day. The longest run is **109 hours**, on both AHUs, from 6 July 09:45 to 10 July 23:05 UTC: the fan never stopped for over four days, so something (probably a schedule override) kept it on. A person reading a chart would find that; the streak trick finds it for every AHU in a building with one query.

The confirmed counts are smaller than the raw counts because the first 5 rows of each full-speed run are not yet “confirmed” (and one-row blips

never are). AHU_1 spends far more time at full speed than AHU_2 (1,349 vs 222 rows), which is itself worth a question.

Try it

1. Change the confirmation from 6 rows to 12 (one hour) in both scripts. In SQL there are two places to change. What happens to the two routes if you only change `:confirm`?
2. Find the longest time each AHU was **off** (status 0) in a row.

Answers

1. With 12 rows, AHU_1 drops to 995 confirmed rows in 21 episodes, and AHU_2 to 160 rows in 5 episodes. The streak route only needs `:confirm` (or `CONFIRM_ROWS`). The window route also needs its frame (`ROWS BETWEEN 11 PRECEDING`), and the SQL check `rows_in_window = :confirm`. Change only the parameter and the frame still holds 6 rows, so the window route finds none (SQL) while the streak route finds the new answer. The two columns disagree, which is how you'd spot the slip. pandas hides it, because `rolling(CONFIRM_ROWS)` reads the same constant.
2. Filter the streaks to `supply_fan_status = 0` instead of `1` and take the longest. It is 999 rows, 83.25 hours, on both AHUs. The streak table already has all the runs of both kinds; only the filter changes.

Why it matters

Day 14 builds an entire fault rule from these pieces: a raw flag, the streak trick, `streak_len >= N`, and a sum of fault rows times five minutes. Day 15 reuses the trick to find flatlined sensors, where the "value" being compared is the sensor reading itself.

day13_pandas.py

```python
"""Day 13, pandas: streaks ("gaps and islands") with the cumsum trick.

    uv run python lessons/unit1-sqlite/day13_pandas.py

Two questions: what is the longest continuous run of the supply fan on each AHU, and when has the
fan been at (nearly) full speed for 30 minutes (6 rows) in a row? Each is a streak problem. The
trick: mark the rows where the value changes, cumulatively sum the marks to number the streaks,
then count rows inside each streak.
A second route to "true for N rows in a row" (a rolling sum) is checked against it.
Compare the printout with day13_sql.py: the numbers should match.
"""

from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

FULL_SPEED_PCT = 95.0  # "at or near full speed"
CONFIRM_ROWS = 6  # 6 rows x 5 min = 30 minutes

# 1. Load: both AHUs, the columns we need, in (equipment, time) order.
COLUMNS = ["timestamp_utc", "supply_fan_status", "supply_fan_speed_pct"]
frames = []
for equip in ("AHU_1", "AHU_2"):
```

```
    df = pd.read_csv(DATA / equip /
         "history_wide.csv", usecols=COLUMNS)
    df.insert(0, "equipment_id", equip)
    frames.append(df)
h = pd.concat(frames, ignore_index=True)
h = h.sort_values(["equipment_id",
         "timestamp_utc"], ignore_index=True)
eq = h["equipment_id"]


def streaks(value):
    """Number the runs of equal consecutive
         values per AHU, and the row position
         inside each run.

    1) new_streak = 1 where the value differs
         from the previous row (an AHU's first
         row starts one);
    2) streak_id = running sum of new_streak,
         so every row in a run shares one id;
    3) streak_len = 1, 2, 3, ... inside each
         (AHU, streak).
    """
    new_streak =
         value.ne(value.groupby(eq).shift()).astype(int)
    streak_id = new_streak.groupby(eq).cumsum()
    streak_len = value.groupby([eq,
         streak_id]).cumcount() + 1
    return streak_id, streak_len


# 2. Fan runs: streaks of supply_fan_status,
         keeping the ones where the fan is on
         (status 1).
run_id, run_len =
         streaks(h["supply_fan_status"])
rows = h[["equipment_id", "timestamp_utc",
         "supply_fan_status"]].assign(
    run_id=run_id, run_len=run_len
)
on = rows[rows["supply_fan_status"] == 1]
fan_runs = (
    on.groupby(["equipment_id", "run_id"])
    .agg(
```

```
        start_utc=("timestamp_utc", "min"),
        end_utc=("timestamp_utc", "max"),
        rows=("run_len", "max"),
    )
    .reset_index()
)
fan_runs["hours"] = fan_runs["rows"] * 5 / 60

# 3. The longest run per AHU (ties: the
        earliest). Sort, then take the first
        row of each group.
longest = (
    fan_runs.sort_values(["equipment_id",
        "rows", "start_utc"], ascending=[True,
        False, True])
    .groupby("equipment_id")
    .head(1)
    .rename(
        columns={"rows": "longest_rows",
        "hours": "longest_hours", "start_utc":
        "longest_start"}
    )
)
run_summary = (
    fan_runs.groupby("equipment_id")
    .agg(runs=("run_id", "size"),
        avg_run_hours=("hours", "mean"))
    .join(longest.set_index("equipment_id")
        [["longest_rows", "longest_hours",
        "longest_start"]])
    .reset_index()
)

# 4. Confirmation: "full speed for 6 rows in a
        row". Same trick, on a 0/1 raw flag.
raw = ((h["supply_fan_status"] == 1) &
        (h["supply_fan_speed_pct"] >=
        FULL_SPEED_PCT)).astype(int)
raw_id, raw_len = streaks(raw)
confirmed_streak = ((raw == 1) & (raw_len >=
        CONFIRM_ROWS)).astype(int)
```

```
# 5. The other route: a rolling sum over the
        last 6 rows equals 6 only if all six
        were 1.
rolling = raw.groupby(eq).transform(lambda s:
        s.rolling(CONFIRM_ROWS).sum())
confirmed_window = (rolling ==
        CONFIRM_ROWS).astype(int)

confirm = h[["equipment_id",
        "timestamp_utc"]].assign(
    raw=raw, streak_id=raw_id,
        streak_len=raw_len,
    confirmed_streak=confirmed_streak,
        confirmed_window=confirmed_window,
)  # fmt: skip
confirm_summary = (
    confirm.groupby("equipment_id")
    .agg(
        raw_rows=("raw", "sum"),
        confirmed_rows_streak=
         ("confirmed_streak", "sum"),
        confirmed_rows_window=
         ("confirmed_window", "sum"),
    )
    .join(
        confirm[confirm["confirmed_streak"] ==
         1]
        .groupby("equipment_id")["streak_id"]
        .nunique()
        .rename("episodes")
    )
    .fillna({"episodes": 0})
    .reset_index()
)

RESULTS = {
    "fan_runs": fan_runs,
    "run_summary": run_summary,
    "confirm_rows": confirm,
    "confirm_summary": confirm_summary,
}

print("Fan runs per AHU (status = 1 streaks):")
```

```
print(run_summary.round(2).to_string(index=False))
print("\nFull speed (>= 95 %) for 6 rows in a
        row, by streak id and by rolling
        sum:")
print(confirm_summary.to_string(index=False))
```

day13_sql.py

```
"""Day 13, SQL: streaks ("gaps and islands")
        with three window-function steps.

    uv run python lessons/course.py setup
        # once: loads the CSVs into
        var/lessons.sqlite
    uv run python lessons/unit1-
        sqlite/day13_sql.py

The same computation as day13_pandas.py, done
        by SQLite on the same rows. SQLite
        does all the work;
pandas only turns the result rows into a table
        for printing.
Compare the printout with day13_pandas.py: the
        numbers should match.
"""

import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT /
        "var" / "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv
        run python lessons/course.py setup")
conn = sqlite3.connect(DB)

PARAMS = {
    "full_speed": 95.0,  # "at or near full
        speed" (percent)
    "confirm": 6,  # 6 rows x 5 min = 30
        minutes
}

# 1. Load: both AHUs, the columns we need.
HISTORY_SQL = """
```

```
CREATE TEMP TABLE h AS
SELECT 'AHU_1' AS equipment_id, timestamp_utc,
         supply_fan_status,
         supply_fan_speed_pct FROM ahu_1
UNION ALL
SELECT 'AHU_2', timestamp_utc,
         supply_fan_status,
         supply_fan_speed_pct FROM ahu_2
"""

# 2. Fan runs. The three-step streak trick,
         once per CTE:
#    a) mark the rows where the value changes
         b) running SUM of the marks = streak
         id
#    c) ROW_NUMBER inside each streak.  The
         first row has LAG NULL, so "= LAG" is
         NULL: ELSE 1.
FAN_ROWS_SQL = """
CREATE TEMP TABLE fan_rows AS
WITH marks AS (
  SELECT equipment_id, timestamp_utc,
         supply_fan_status,
         CASE WHEN supply_fan_status =
         LAG(supply_fan_status) OVER w THEN 0
         ELSE 1 END AS new_streak
  FROM h
  WINDOW w AS (PARTITION BY equipment_id ORDER
         BY timestamp_utc)
), ids AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY
         equipment_id ORDER BY timestamp_utc)
         AS run_id
  FROM marks
)
SELECT equipment_id, timestamp_utc,
         supply_fan_status, run_id,
       ROW_NUMBER() OVER (PARTITION BY
         equipment_id, run_id ORDER BY
         timestamp_utc) AS run_len
FROM ids
"""
FAN_RUNS_SQL = """
CREATE TEMP TABLE fan_runs AS
```

```
SELECT equipment_id, run_id,
       min(timestamp_utc) AS start_utc,
         max(timestamp_utc) AS end_utc,
       count(*) AS rows, count(*) * 5 / 60.0 AS
         hours
FROM fan_rows
WHERE supply_fan_status = 1
GROUP BY equipment_id, run_id
"""
# 3. The longest run per AHU (ties: the
         earliest): rank the runs, keep rank 1.
RUN_SUMMARY_SQL = """
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY
         equipment_id ORDER BY rows DESC,
         start_utc) AS rank
  FROM fan_runs
)
SELECT equipment_id,
       (SELECT count(*) FROM fan_runs r WHERE
         r.equipment_id = k.equipment_id) AS
         runs,
       (SELECT avg(hours) FROM fan_runs r WHERE
         r.equipment_id = k.equipment_id) AS
         avg_run_hours,
       rows AS longest_rows, hours AS
         longest_hours, start_utc AS
         longest_start
FROM ranked k
WHERE rank = 1
ORDER BY equipment_id
"""

# 4. Confirmation: full speed for 6 rows in a
         row. Same trick on a 0/1 raw flag...
# 5. ...and the other route: SUM over the last
         6 rows equals 6 only if all six were
         1.
CONFIRM_SQL = """
CREATE TEMP TABLE confirm AS
WITH base AS (
  SELECT equipment_id, timestamp_utc,
         CASE WHEN supply_fan_status = 1 AND
         supply_fan_speed_pct >= :full_speed
```

```
            THEN 1 ELSE 0 END AS raw
  FROM h
), marks AS (
  SELECT *, CASE WHEN raw = LAG(raw) OVER w
         THEN 0 ELSE 1 END AS new_streak,
         SUM(raw) OVER (PARTITION BY
         equipment_id ORDER BY timestamp_utc
                        ROWS BETWEEN 5
         PRECEDING AND CURRENT ROW) AS last6,
         COUNT(*) OVER (PARTITION BY
         equipment_id ORDER BY timestamp_utc
                        ROWS BETWEEN 5
         PRECEDING AND CURRENT ROW) AS
         rows_in_window
  FROM base
  WINDOW w AS (PARTITION BY equipment_id ORDER
         BY timestamp_utc)
), ids AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY
         equipment_id ORDER BY timestamp_utc)
         AS streak_id
  FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY
         equipment_id, streak_id ORDER BY
         timestamp_utc)
            AS streak_len
  FROM ids
)
SELECT equipment_id, timestamp_utc, raw,
         streak_id, streak_len,
       CASE WHEN raw = 1 AND streak_len >=
         :confirm THEN 1 ELSE 0 END AS
         confirmed_streak,
       CASE WHEN rows_in_window = :confirm AND
         last6 = :confirm
            THEN 1 ELSE 0 END AS
         confirmed_window
FROM ranked
"""
CONFIRM_SUMMARY_SQL = """
SELECT equipment_id,
       sum(raw) AS raw_rows,
```

```
        sum(confirmed_streak) AS
          confirmed_rows_streak,
        sum(confirmed_window) AS
          confirmed_rows_window,
        count(DISTINCT CASE WHEN
          confirmed_streak = 1 THEN streak_id
          END) AS episodes
FROM confirm
GROUP BY equipment_id
ORDER BY equipment_id
"""

conn.execute(HISTORY_SQL)
conn.execute(FAN_ROWS_SQL)
conn.execute(FAN_RUNS_SQL)
conn.execute(CONFIRM_SQL, PARAMS)

fan_runs = pd.read_sql_query("SELECT * FROM
          fan_runs ORDER BY equipment_id,
          run_id", conn)
run_summary =
          pd.read_sql_query(RUN_SUMMARY_SQL,
          conn)
confirm = pd.read_sql_query("SELECT * FROM
          confirm ORDER BY equipment_id,
          timestamp_utc", conn)
confirm_summary =
          pd.read_sql_query(CONFIRM_SUMMARY_SQL,
          conn)

RESULTS = {
    "fan_runs": fan_runs,
    "run_summary": run_summary,
    "confirm_rows": confirm,
    "confirm_summary": confirm_summary,
}

print("Fan runs per AHU (status = 1 streaks):")
print(run_summary.round(2).to_string(index=False))
print("\nFull speed (>= 95 %) for 6 rows in a
          row, by streak id and by rolling
          sum:")
print(confirm_summary.to_string(index=False))
```

Day 14 — Write a fault rule (FC1, duct static too low) twice, and check it against Open-FDD

Goal: turn an engineering rule into a pandas script *and* a SQL script, see that they give the same answer, count fault hours, and compare with a production FDD engine's results.

Read

Fault detection and diagnostics (FDD) runs rules over trend data to find equipment that isn't doing what it should. Open-FDD's rules come from ASHRAE Guideline 36 and the NIST/PNNL APAR rules. **FC1** says:

> The supply fan is running (at or near full speed) but duct static pressure stays below its setpoint.

The fan is working as hard as it can and still can't make pressure. Likely causes: a broken belt, a failing VFD, a blocked filter or coil, a leaky duct, or a setpoint too high for this system.

Every Open-FDD rule (`sql/fdd/postgres/rules/*.sql`, and its pandas twin in `open_fdd/rules`) has the same shape:

1. **Gate**: only judge rows where the equipment is on (`supply_fan_status = 1`). Off-hours aren't faults.
2. **Raw fault**: `CASE WHEN <condition> THEN 1 ELSE 0 END`, with a tolerance so noise doesn't trip it.
3. **Confirm**: count a row only once the raw fault has held for N consecutive rows (Day 13's streaks).
4. **Aggregate**: fault rows × 5 minutes = fault hours.

The math is **SI**, like Open-FDD (Day 6): the CSV's `da_p_inwc` is in inches of water, so both scripts convert to pascals first. The tolerance is **25 Pa** (about 0.1 inH2O) and a fault is confirmed after **2 rows** (10 minutes).

The inputs are **hard-coded column names**: `supply_fan_status`, `da_p_inwc`, `da_p_setpoint_inwc`. Both AHUs' exports happen to use the same names, so one script covers both. That's fine for one building. Week 5 shows where it breaks, and how a data model replaces those names.

Open-FDD keeps every rule twice on purpose. SQL is what runs next to the data. pandas is the **oracle**: a second, independent implementation, easy to step through in a notebook. Its parity tests run both on the same data and fail if a single row differs. Today you write both halves and compare them.

How each step maps across:

Step	`day14_sql.py` (per row, window functions)	`day14_pandas.py` (per vectorized)
load both AHUs	`SELECT ... FROM ahu_1 UNION ALL SELECT ... FROM ahu_2`	`pd.concat([read_csv(AH read_csv(AHU_2)])`
SI	`da_p_inwc * 249.08891 AS duct_static_pa`	`h["da_p_inwc"] * 249.0`
gate + raw fault	`CASE WHEN supply_fan_status = 1 AND ... THEN 1 ELSE 0 END`	`(h.supply_fan_status = (...)`
"value changed"	`raw = LAG(raw) OVER (PARTITION BY equipment_id ORDER BY ts)`	`raw.ne(raw.groupby(eq)`
streak id	`SUM(new_streak) OVER (... ORDER BY ts)`	`new.groupby(eq).cumsur`
row within streak	`ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id ...)`	`groupby([eq, streak]).cumcount() +`
hours	`sum(fault) * 5 / 60.0`	`fault.sum() * 5 / 60`

Both are O(n) once the rows are in `(equipment_id, time)` order: SQLite sorts for the window, pandas sorts with `sort_values`. `groupby(eq)` in pandas plays the part of `PARTITION BY equipment_id`.

Run

```
uv run python lessons/course.py run 14
```

That runs `day14_pandas.py`, then `day14_sql.py`. Read them side by side: each numbered comment in one has the same number in the other. Both end by printing:

```
FC1 per AHU (hours, episodes):
equipment_id  raw_fault_h  confirmed_fault_h
episodes
       AHU_1       120.00             114.25
53
       AHU_2         3.42               1.50
5
```

AHU_1 has about 114 confirmed fault hours in 53 episodes. AHU_2 has 1.5 hours: essentially healthy for this rule (its problem is the opposite, static *above* setpoint, as Day 8 showed).

Notice the hours are printed with two decimals. AHU_1 has 1,371 confirmed rows = 114.25 h exactly, a tie at one decimal. SQLite's `round(114.25, 1)` rounds halves away from zero (114.3), numpy and Python round halves to even (114.2). Round only for display, never before comparing two results.

Compare with Open-FDD. Its results for this data ship next to the CSVs in `data/BUILDING_50/*/fdd_events.csv` (the SQL side has them as the `fdd_events` table). It found **80 FC1 episodes, 484 hours** on AHU_1, against our 53 and 114. Different numbers don't mean either is

wrong. Check whether the two engines are finding **the same events**: how many of Open-FDD's episodes contain at least one of our confirmed fault rows?

- SQL uses a **correlated `EXISTS`**: for each of the E events, probe our fault rows for that AHU and time range. With an index on `(equipment_id, timestamp_utc)` each probe is a B-tree range seek, so the whole thing is O(E log N).
- pandas has no `EXISTS`. The fast equivalent is **binary search**: sort our fault timestamps once, then `np.searchsorted` finds for every event at once how many fault rows fall before its start and at or before its end. If those differ, the event contains one. Same O(E log F), written as array code.

Both print **73 of 80**. Same problems, measured differently. Open-FDD's episode clock runs from first to last fault sample, including stretches in between, and its event timestamps aren't on this 5-minute grid. It ran on its own copy of the data with its own tuning. Totals depend on those choices; overlap tells you whether the rules agree on **what** is wrong.

Real Open-FDD rules (`cmd1_fan_mismatch.sql` is a short one to read) add one more step: **forward sample-and-hold with a gap limit** (`MAX_GAP_HOURS`). Building 50 has no gaps (every step is exactly 5 minutes), so counting rows works here. Day 15 checks that, and Day 18 builds the gap-aware version.

Try it

1. Change the tolerance from 25 Pa to 50 Pa (~0.2 inH2O) in both scripts (`TOL_PA` in pandas, `"tol"` in SQL). How much do AHU_1's confirmed hours drop? Do the two scripts still agree?
2. Write the opposite rule, **duct static too high** (`duct_static > setpoint + 25 Pa` while running), in both scripts, and run it on AHU_2.

Answers

1. From about 114 to about 101 hours (101.2). Most of AHU_1's shortfall is more than 50 Pa: a real problem, not a marginal one.
2. Flip one comparison in each script: `h["duct_static_pa"] > h["duct_static_sp_pa"] + TOL_PA` and `duct_static_pa > duct_static_sp_pa + :tol`. AHU_2 shows about 1,428 confirmed hours, nearly all of its 1,500 run hours. The fan pushes far more pressure than its setpoint asks for, all the time: wasted fan energy, and possibly a reset sequence that isn't working. Open-FDD's version of this rule is `ahu_ducthi.sql`.

Why it matters

Notice what the rule needed from outside the data: which column is "duct static" and which is "fan status" for each AHU. Here those names are typed into the code. Change buildings, or meet an AHU whose export names a column differently, and the rule breaks even though the physics hasn't changed. Week 5 fixes that with a semantic model: the rule

stays the same, and the model tells it where its inputs are. And because you have the rule twice, a change to either script that alters a single row shows up as a mismatch (`tests/test_lesson_scripts.py` checks every day).

day14_pandas.py

```python
"""Day 14, pandas: FC1 (duct static too low while the fan runs) on both AHUs.

    uv run python lessons/unit1-sqlite/day14_pandas.py

Reads the historian's CSV exports straight into DataFrames. Column names are hard-coded on purpose:
this is how a first FDD script looks. (Week 5 replaces them with a model lookup.)
Compare the printout with day14_sql.py: the numbers should match.
"""

from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

PA_PER_INH2O = 249.08891  # the historian logged inches of water; fault math is SI (pascals)
TOL_PA = 25.0  # how far below setpoint counts as a fault (~0.1 inH2O)
CONFIRM_ROWS = 2  # 2 rows x 5 min = 10 minutes before a fault counts

# 1. Load: one CSV per AHU, only the columns FC1 needs. Same column names on both AHUs.
COLUMNS = ["timestamp_utc", "supply_fan_status", "da_p_inwc", "da_p_setpoint_inwc"]
frames = []
for equip in ("AHU_1", "AHU_2"):
    df = pd.read_csv(DATA / equip / "history_wide.csv", usecols=COLUMNS)
```

```
    df.insert(0, "equipment_id", equip)
    frames.append(df)
h = pd.concat(frames, ignore_index=True)
h = h.sort_values(["equipment_id",
        "timestamp_utc"], ignore_index=True)

# 2. SI units
h["duct_static_pa"] = h["da_p_inwc"] *
        PA_PER_INH2O
h["duct_static_sp_pa"] =
        h["da_p_setpoint_inwc"] * PA_PER_INH2O

# 3. Gate + raw fault: fan running and static
        more than TOL_PA below setpoint
eq = h["equipment_id"]
raw = (h["supply_fan_status"] == 1) &
        (h["duct_static_pa"] <
        h["duct_static_sp_pa"] - TOL_PA)
raw = raw.astype(int)

# 4. Confirm: number the streaks (runs of equal
        raw values), then the rows inside each
        streak
new_streak =
        raw.ne(raw.groupby(eq).shift()).astype(int)
        # first row of each AHU starts a streak
streak_id = new_streak.groupby(eq).cumsum()
streak_len = raw.groupby([eq,
        streak_id]).cumcount() + 1
fault = ((raw == 1) & (streak_len >=
        CONFIRM_ROWS)).astype(int)
fc1 = h[["equipment_id",
        "timestamp_utc"]].assign(raw_fault=raw,
        streak_id=streak_id, fault=fault)

# 5. Aggregate: rows x 5 minutes = hours, and
        confirmed episodes
hours = fc1.groupby("equipment_id")
        [["raw_fault", "fault"]].sum() * 5 /
        60
episodes = fc1[fc1["fault"] ==
        1].groupby("equipment_id")
        ["streak_id"].nunique()
summary = (
```

```
    hours.rename(columns={"raw_fault":
         "raw_fault_h", "fault":
         "confirmed_fault_h"})
    .assign(episodes=episodes)
    .fillna({"episodes": 0})
    .reset_index()
)

# 6. Compare with Open-FDD: its FC1 episodes
         for this data, and how many contain
         one of our faults
events = pd.concat(
    [pd.read_csv(DATA / e / "fdd_events.csv")
         for e in ("AHU_1", "AHU_2")],
         ignore_index=True
)
events = events[events["rule_name"] ==
         "fc1"].copy()
events["equipment_id"] =
         events["hvac_system"].str.replace("bld_50_",
         "", regex=False)
openfdd = (
    events.groupby("equipment_id")
    .agg(episodes=("rule_name", "size"), hours=
         ("duration_min", "sum"))
    .assign(hours=lambda d: d["hours"] / 60)
    .reset_index()
)

found = np.zeros(len(events), dtype=bool)
for equip, ev in
         events.groupby("equipment_id"):
    # binary search: sorted fault times; an
         event contains a fault if one falls in
         [start, end]
    times = np.sort(
        fc1.loc[(fc1.equipment_id == equip) &
         (fc1.fault == 1),
         "timestamp_utc"].to_numpy()
    )
    lo = np.searchsorted(times,
         ev["start_time_utc"].to_numpy(),
         side="left")
```

```
    hi = np.searchsorted(times,
         ev["end_time_utc"].to_numpy(),
         side="right")
    found[events.index.get_indexer(ev.index)] = hi
         > lo
overlap = pd.DataFrame({"openfdd_episodes":
         [len(events)], "also_found_by_us":
         [int(found.sum())]})

RESULTS = {"fc1_per_row": fc1, "fc1_summary":
         summary, "openfdd_fc1": openfdd,
         "overlap": overlap}

print("FC1 per AHU (hours, episodes):")
print(summary.round(2).to_string(index=False))
print("\nOpen-FDD's FC1 episodes for the same
         data:")
print(openfdd.round(1).to_string(index=False))
print("\nOpen-FDD episodes that contain at
         least one of our confirmed fault
         rows:")
print(overlap.to_string(index=False))
```

day14_sql.py

```
"""Day 14, SQL: FC1 (duct static too low while
        the fan runs) on both AHUs.

    uv run python lessons/course.py setup
        # once: loads the CSVs into
        var/lessons.sqlite
    uv run python lessons/unit1-
        sqlite/day14_sql.py

The same computation as day14_pandas.py, done
        by SQLite on the same rows. SQLite
        does all the work;
pandas only turns the result rows into a table
        for printing.
Compare the printout with day14_pandas.py: the
        numbers should match.
"""

import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT /
        "var" / "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv
        run python lessons/course.py setup")
conn = sqlite3.connect(DB)

PARAMS = {
    "pa_per_inh2o": 249.08891,  # the historian
        logged inches of water; fault math is
        SI (pascals)
    "tol": 25.0,  # how far below setpoint
        counts as a fault (~0.1 inH2O)
    "confirm": 2,  # 2 rows x 5 min = 10
        minutes before a fault counts
}
```

```
# One query, one CTE per step. ahu_1 and ahu_2
        are the CSVs loaded as tables, same
        column names.
FC1_SQL = """
CREATE TEMP TABLE fc1 AS
WITH h AS (                                      -
        - 1. load: both AHUs, the columns FC1
        needs
  SELECT 'AHU_1' AS equipment_id,
        timestamp_utc, supply_fan_status,
        da_p_inwc, da_p_setpoint_inwc
  FROM ahu_1
  UNION ALL
  SELECT 'AHU_2', timestamp_utc,
        supply_fan_status, da_p_inwc,
        da_p_setpoint_inwc
  FROM ahu_2
), si AS (                                       -
        - 2. SI units
  SELECT equipment_id, timestamp_utc,
        supply_fan_status,
        da_p_inwc * :pa_per_inh2o          AS
        duct_static_pa,
        da_p_setpoint_inwc * :pa_per_inh2o AS
        duct_static_sp_pa
  FROM h
), base AS (                                     -
        - 3. gate + raw fault
  SELECT equipment_id, timestamp_utc,
        CASE WHEN supply_fan_status = 1 AND
        duct_static_pa < duct_static_sp_pa -
        :tol
             THEN 1 ELSE 0 END AS raw_fault
  FROM si
), marks AS (                                    -
        - 4a. a new streak starts where
        raw_fault changes
  SELECT *, CASE WHEN raw_fault =
        LAG(raw_fault) OVER w THEN 0 ELSE 1
        END AS new_streak
  FROM base
  WINDOW w AS (PARTITION BY equipment_id ORDER
        BY timestamp_utc)
```

```
), streaks AS (                                  -
         - 4b. running sum of starts numbers
         the streaks
  SELECT *, SUM(new_streak) OVER (PARTITION BY
         equipment_id ORDER BY timestamp_utc)
         AS streak_id
  FROM marks
), ranked AS (                                   -
         - 4c. row number inside each streak
  SELECT *, ROW_NUMBER() OVER (PARTITION BY
         equipment_id, streak_id ORDER BY
         timestamp_utc)
            AS streak_len
  FROM streaks
)
SELECT equipment_id, timestamp_utc, raw_fault,
         streak_id,
       CASE WHEN raw_fault = 1 AND streak_len
         >= :confirm THEN 1 ELSE 0 END AS fault
FROM ranked
"""
conn.execute(FC1_SQL, PARAMS)
conn.execute("CREATE INDEX temp.fc1_eq_ts ON
         fc1 (equipment_id, timestamp_utc)")

SUMMARY_SQL = """
SELECT equipment_id,                             -
         - 5. rows x 5 minutes = hours
       sum(raw_fault) * 5 / 60.0 AS
         raw_fault_h,
       sum(fault) * 5 / 60.0     AS
         confirmed_fault_h,
       count(DISTINCT CASE WHEN fault = 1 THEN
         streak_id END) AS episodes
FROM fc1
GROUP BY equipment_id
ORDER BY equipment_id
"""

OPENFDD_SQL = """
SELECT replace(hvac_system, 'bld_50_', '') AS
         equipment_id,
       count(*) AS episodes, sum(duration_min)
         / 60 AS hours
```

```
FROM fdd_events
WHERE rule_name = 'fc1'
GROUP BY equipment_id
"""

# 6. For each Open-FDD episode, does at least
     one of our confirmed fault rows fall
     inside it?
# The timestamps are ISO text in one format, so
     BETWEEN on text compares them in time
     order.
OVERLAP_SQL = """
SELECT count(*) AS openfdd_episodes,
       sum(EXISTS (SELECT 1 FROM fc1 f
                   WHERE f.equipment_id =
         replace(e.hvac_system, 'bld_50_', '')
                     AND f.fault = 1
                     AND f.timestamp_utc
         BETWEEN e.start_time_utc AND
         e.end_time_utc))
         AS also_found_by_us
FROM fdd_events e
WHERE e.rule_name = 'fc1'
"""

fc1 = pd.read_sql_query("SELECT * FROM fc1
     ORDER BY equipment_id, timestamp_utc",
     conn)
summary = pd.read_sql_query(SUMMARY_SQL, conn)
openfdd = pd.read_sql_query(OPENFDD_SQL, conn)
overlap = pd.read_sql_query(OVERLAP_SQL, conn)

RESULTS = {"fc1_per_row": fc1, "fc1_summary":
     summary, "openfdd_fc1": openfdd,
     "overlap": overlap}

print("FC1 per AHU (hours, episodes):")
print(summary.round(2).to_string(index=False))
print("\nOpen-FDD's FC1 episodes for the same
     data:")
print(openfdd.round(1).to_string(index=False))
```

```
print("\nOpen-FDD episodes that contain at
        least one of our confirmed fault
        rows:")
print(overlap.to_string(index=False))
```

Day 15 — Data quality before you trust a rule, and the Unit 1 challenge

Goal: check the data before judging the equipment: dropouts, a setpoint that reads zero, out-of-range values, flatlined sensors, the time step, and coverage. Then finish Unit 1 by writing a discharge air temperature rule in both pandas and SQL.

Read

A fault rule can only be as good as its inputs. A rule that fires on a sensor dropout reports a broken *logger*, not a broken *air handler*, and the person who gets the work order can't tell the difference. So before a rule, check the data. Today's checks all run on both AHUs, in both scripts, with the same answers. They are `day15_pandas.py` and `day15_sql.py`.

Check	What it looks for	pandas	
dropout	a temperature that reads exactly 0 °F	`(h.discharge_air_temp_f == 0) | ...`	`CAS dis = 0`
missing setpoint	setpoint reads 0 while the fan runs	`running & (h.dat_reset_f == 0)`	`sup 1 A 0`

Check	What it looks for	pandas	
out of range	outside what a sensor can report	`(x < lo) | (x > hi)`	`x N AND`
flatline	an identical value for 1 h or more	`streaks(value)` (Day 13)	the (Da
time step	every row 300 s after the last	`epoch.groupby(eq).diff()`	`uni LAG OVE`
coverage	rows present vs rows expected	`groupby([eq, month]).agg(...)`	`GRO equ`

All temperatures are converted to °C (`(°F − 32) × 5/9`) and static to Pa before the checks, like Day 14. The dropout test is the one place where we look at the raw °F value on purpose.

Dropouts, and the comparison trap

Seven rows on each AHU have the mixed, return and discharge temperatures all reading **exactly 0 °F**. A logger writes 0 when it gets no reading; 0 °F is a perfectly valid number, so nothing flags it. After conversion that 0 °F is **-17.78 °C**, and that is where the trap is.

```
(h.discharge_air_temp_f == 0)  # 7 rows: the
        raw value is exactly 0.0
(q.dat_c == -17.78)  # 0 rows: the converted
        value is -17.7777777...
(q.dat_c - ZERO_F_IN_C).abs() < 0.01  # 7 rows:
        compare with a tolerance
```

-17.78 is a rounded, printed number. The value in memory is -17.777777... and floating-point `==` against any rounded constant never matches. Two safe ways: test the **raw value** before converting (exact zeros are stored exactly), or compare converted values with a **tolerance**. Never `==` a float you computed. The script prints both the exact-match count (0) and the tolerance count (7) next to each other.

The seven rows form three episodes: one row on 31 March, five on 22 April (16:10 to 16:30 UTC), one on 15 June. Notice that they are not caught by the **range check**: our sensor range is -40 to 60 °C, and -17.78 °C is inside it. A range check catches impossible values (a duct at 500 °C); a dropout is a *plausible wrong* value, which is harder. Out-of-range is 0 rows here. That is a result too.

A setpoint of zero, and flatlines

`dat_reset_f` is the discharge air temperature **setpoint**. While the fan runs it should sit between 50 and 71 °F. It reads **0 °F** for 1,233 running rows on AHU_1 and 984 on AHU_2, mostly in June. Seven of those are the dropout rows. The rest are the control system publishing no setpoint. A rule

comparing the discharge temperature with a setpoint of -17.78 °C would call the whole stretch a fault.

Flatlines. A working temperature sensor wiggles in its last digits. The same value for an hour or more while the fan runs is suspicious. This is Day 13's streak trick on the sensor reading: streaks of an identical `discharge_air_temp_f`, kept if the streak is at least 12 rows (1 hour) and the fan ran for all of it. That finds 66 episodes (1,333 rows) on AHU_1 and 48 (1,064 rows) on AHU_2, the longest 136 and 135 rows (11 hours). Remember Day 12: the historian logs change-of-value, so a stable reading is *normally* repeated. Treat flatlines as "look here", not as proof, and raise the threshold if it's too chatty (see Try it).

The time step, and coverage

Every row should be exactly 300 s after the one before. The check takes the difference to the previous timestamp and counts rows where it isn't 300: **0 gaps, max step 300 s** on both AHUs. That is why counting rows works as counting time in Days 12 to 14 (`ROWS BETWEEN 11 PRECEDING` really is one hour). Real data usually has gaps. Day 18 builds the gap-aware version.

Coverage per AHU per month compares the rows present with the rows expected from the first to the last timestamp in the month: 100% for every month here. March and July are partial months

(the export starts on 16 March and ends on 17 July), so compare coverage against the *span*, not against a calendar month.

Run

```
uv run python lessons/course.py run 15
```

Both scripts print the same numbers (pandas shows `300.0` where SQL shows `300`). A flavour of it:

```
Quality checks per AHU:
equipment_id               AHU_1  AHU_2
dropout_rows                   7      7
sp_zero_rows                1233    984
out_of_range_rows              0      0
flatline_rows               1333   1064
step_gaps                      0      0
max_step_s                   300    300
flat_episodes                 66     48
dat_eq_minus_17_78             0      0
dat_within_0_01_of_zero_f      7      7
```

followed by the seven dropout rows, a coverage table, and the challenge answer below.

Try it

1. Raise the flatline threshold from 12 rows to 36 (3 hours), in both scripts (`FLAT_ROWS` in pandas, `flat_rows` in SQL). How many episodes are left?
2. The `dropout` flag looks at three temperatures. Why is it safer to flag the *row* than to remove the one dropout sensor?

Answers

1. Five episodes remain on each AHU (419 rows on AHU_1, 416 on AHU_2). Most hour-long flatlines are probably change-of-value repeats. The few stretches of 3 hours or more are the ones worth asking about.
2. In a dropout all three sensors fail at the same moment, which points at the controller or the network rather than one sensor. A rule that mixes several temperatures would otherwise compute with a mix of real and fake values. Flag the whole row and let each rule skip it.

Unit 1 challenge: a discharge air temperature rule, in SI, with a quality gate

Write the rule in both scripts (it's already in both, as step 10, so the parity test covers it; try writing it yourself first):

> While the supply fan runs, the discharge air temperature is **more than 2.78 °C (5 °F) away from its setpoint** (`discharge_air_temp_f` vs `dat_reset_f`), and this holds for **30 minutes (6 rows)**.

Report **fault hours per AHU per month**. Do it twice: once trusting every row, once skipping rows where the data quality checks fail.

Answer

Same shape as Day 14: gate, raw fault, confirm, aggregate.

```
err_c = (q.dat_c - q.dat_sp_c).abs()  # both
        converted: (F - 32) * 5 / 9
raw = running & (err_c > 2.78)  # ungated
raw_gated = running & (q.dropout == 0) &
        (q.sp_zero == 0) & (err_c > 2.78)
# then Day 13's streak trick per AHU: fault =
        raw & (streak_len >= 6); hours = rows
        * 5 / 60
```

```
CASE WHEN supply_fan_status = 1 AND dropout = 0
        AND sp_zero = 0 AND abs(dat_c -
        dat_sp_c) > :tol
     THEN 1 ELSE 0 END AS raw_fault
-- LAG / SUM OVER / ROW_NUMBER as on Day 13,
        then sum(fault) * 5 / 60.0 per
        equipment_id and month
```

The absolute value matters: the discharge temperature can be too high or too low. A *difference* of temperatures converts with only the 5/9 factor (the 32 cancels), so 5 °F is 5 × 5/9 = 2.78 °C. The scripts convert each temperature first and then subtract, which gives the same answer and keeps every column in °C.

Fault hours (ungated / gated):

```
equipment_id   month  ungated_fault_h
gated_fault_h
       AHU_1 2026-03            22.25
22.25
       AHU_1 2026-04            46.00
46.00
       AHU_1 2026-05           113.50
113.50
       AHU_1 2026-06           213.75
123.75
       AHU_1 2026-07            21.00
11.33
       AHU_2 2026-03             3.58
3.58
       AHU_2 2026-04            27.33
27.33
       AHU_2 2026-05            72.67
72.67
       AHU_2 2026-06           174.75
104.67
       AHU_2 2026-07            19.67
10.00
Totals: AHU_1 416.50 -> 316.83 h, AHU_2 298.00
-> 218.25 h
```

Both AHUs miss their discharge setpoint for a good part of the time, most of all in June. The quality gate matters: the missing setpoint adds about 100 hours on AHU_1 and 80 on AHU_2, roughly a quarter of the ungated totals, almost all of it in June and July, because a setpoint of -17.78 °C is “wrong” by 30 °C whenever the fan is on. Those are not faults in the air handler. March and April are unchanged: the 7 dropout rows are too few and too short to reach the 6-row confirmation. Fixing the setpoint publisher is its own work order.

Why it matters

The quality gate changed the answer by about a quarter. That's the point of the day: **read the data before you read the rule's output**. Production FDD does the same checks on every run. The gap-aware sample-and-hold (`MAX_GAP_HOURS`) comes on Day 18.

What production does differently (Timescale). Everything this unit used (CSV, SQLite) is one machine and a few hundred thousand rows. Open-FDD's production stack keeps history in **PostgreSQL with the TimescaleDB extension**: the same SQL (window functions, `CASE`, `GROUP BY` all carry over), but the table is split into time chunks, old chunks compress, and queries for one series and one time range touch only their chunks. That is Day 11's index idea, applied automatically. Hard-coded column names are the other thing that changes: Week 5 replaces `discharge_air_temp_f` with a lookup in a building model, so one rule runs over many buildings and exports.

day15_pandas.py

```python
"""Day 15, pandas: data quality before you trust a rule, then the Unit 1 challenge (a DAT rule).

    uv run python lessons/unit1-sqlite/day15_pandas.py

Checks on both AHUs: dropouts (temperatures that read exactly 0 degF), a setpoint that reads 0
while the fan runs, out-of-range values, flatlined sensors (the streak trick from Day 13), the
5-minute time step, and coverage per month. Then the rule: |discharge air temp - setpoint| >
2.78 degC (5 degF) while the fan runs, confirmed for 30 minutes (6 rows), with and without the
quality gate.
Compare the printout with day15_sql.py: the numbers should match.
"""

from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

PA_PER_INH2O = 249.08891  # the historian logged inches of water; the math is SI (pascals)
FLAT_ROWS = 12  # an identical value for 12 rows x 5 min = 1 hour while the fan runs is suspicious
TOL_C = 2.78  # |discharge - setpoint| above this is a fault (5 degF is 2.78 degC)
CONFIRM_ROWS = 6  # 6 rows x 5 min = 30 minutes
STEP_S = 300  # the historian logs every 5 minutes
```

```
ZERO_F_IN_C = (0 - 32.0) * 5.0 / 9.0  # what a
        dropout looks like after conversion:
        -17.777...
TEMP_RANGE_C = (-40.0, 60.0)  # a sensor's
        physical range
STATIC_RANGE_PA = (0.0, 1500.0)
SPEED_RANGE_PCT = (0.0, 100.0)

# 1. Load: both AHUs, the columns we need, in
        (equipment, time) order.
COLUMNS = [
    "timestamp_utc",
    "supply_fan_status",
    "supply_fan_speed_pct",
    "da_p_inwc",
    "discharge_air_temp_f",
    "dat_reset_f",
    "mixed_air_temp_f",
    "return_air_temp_f",
    "outside_air_temp_f",
]
frames = []
for equip in ("AHU_1", "AHU_2"):
    df = pd.read_csv(DATA / equip /
        "history_wide.csv", usecols=COLUMNS)
    df.insert(0, "equipment_id", equip)
    frames.append(df)
h = pd.concat(frames, ignore_index=True)
h = h.sort_values(["equipment_id",
        "timestamp_utc"], ignore_index=True)
eq = h["equipment_id"]
running = h["supply_fan_status"] == 1


def f_to_c(f):
    return (f - 32.0) * 5.0 / 9.0


def streaks(value):
    """Day 13's trick: the streak id (runs of
        equal values per AHU) and the row
        number inside it."""
```

```
    new_streak =
        value.ne(value.groupby(eq).shift()).astype(int)
    streak_id = new_streak.groupby(eq).cumsum()
    return streak_id, value.groupby([eq,
        streak_id]).cumcount() + 1


# 2. SI units. Keep the raw degF columns too:
        the dropout test must look at the raw
        value.
q = h[["equipment_id", "timestamp_utc",
        "supply_fan_status"]].copy()
q["dat_c"] = f_to_c(h["discharge_air_temp_f"])
q["dat_sp_c"] = f_to_c(h["dat_reset_f"])
q["mat_c"] = f_to_c(h["mixed_air_temp_f"])
q["rat_c"] = f_to_c(h["return_air_temp_f"])
q["oat_c"] = f_to_c(h["outside_air_temp_f"])
q["duct_static_pa"] = h["da_p_inwc"] *
        PA_PER_INH2O

# 3. Dropouts: a duct temperature that reads
        exactly 0 degF (-17.78 degC). Test the
        RAW value.
q["dropout"] = (
    (h["discharge_air_temp_f"] == 0) |
        (h["mixed_air_temp_f"] == 0) |
        (h["return_air_temp_f"] == 0)
).astype(int)
# The comparison trap: the converted value is
        -17.7777..., so == -17.78 never
        matches anything.
trap_exact = (q["dat_c"] == -17.78).astype(int)
trap_tolerance = ((q["dat_c"] -
        ZERO_F_IN_C).abs() < 0.01).astype(int)

# 4. A setpoint of 0 degF while the fan runs is
        not a real setpoint (a missing value,
        shown as 0).
q["sp_zero"] = (running & (h["dat_reset_f"] ==
        0)).astype(int)

# 5. Out of range: outside what the sensor can
        physically report.
temps = q[["dat_c", "mat_c", "rat_c", "oat_c"]]
```

```
bad_temp = ((temps < TEMP_RANGE_C[0]) | (temps
          > TEMP_RANGE_C[1])).any(axis=1)
bad_static = (q["duct_static_pa"] <
          STATIC_RANGE_PA[0]) |
          (q["duct_static_pa"] >
          STATIC_RANGE_PA[1])
bad_speed = (h["supply_fan_speed_pct"] <
          SPEED_RANGE_PCT[0]) | (
    h["supply_fan_speed_pct"] >
          SPEED_RANGE_PCT[1]
)
q["out_of_range"] = (bad_temp | bad_static |
          bad_speed).astype(int)

# 6. Flatlines: streaks of an identical
          discharge temperature, 1 h+ with the
          fan running throughout.
flat_id, q["flat_len"] =
          streaks(h["discharge_air_temp_f"])
island = h.groupby([eq, flat_id])
          ["supply_fan_status"]
island_rows = island.transform("size")
island_running = island.transform("sum")
q["flatline"] = ((island_rows >= FLAT_ROWS) &
          (island_running ==
          island_rows)).astype(int)
flat_episodes = (
    q.assign(flat_id=flat_id)[q["flatline"] ==
          1]
    .groupby(["equipment_id", "flat_id"])
    .agg(start_utc=("timestamp_utc", "min"),
          rows=("flat_len", "max"), value_c=
          ("dat_c", "first"))
    .reset_index()
)

# 7. Time step: seconds since the previous row
          of the same AHU. Every step should be
          300.
ts = pd.to_datetime(h["timestamp_utc"],
          utc=True)
epoch_s = (ts - pd.Timestamp("1970-01-01",
          tz="UTC")).dt.total_seconds()
q["step_s"] = epoch_s.groupby(eq).diff()
q["epoch_s"] = epoch_s
```

```
# 8. One line of checks per AHU.
checks = q.groupby("equipment_id").agg(
    dropout_rows=("dropout", "sum"),
    sp_zero_rows=("sp_zero", "sum"),
    out_of_range_rows=("out_of_range", "sum"),
    flatline_rows=("flatline", "sum"),
    step_gaps=("step_s", lambda s: int((s !=
        STEP_S).sum() - s.isna().sum())),
    max_step_s=("step_s", "max"),
)
checks["flat_episodes"] =
        flat_episodes.groupby("equipment_id").size()
checks["dat_eq_minus_17_78"] =
        trap_exact.groupby(eq).sum()
checks["dat_within_0_01_of_zero_f"] =
        trap_tolerance.groupby(eq).sum()
checks = checks.fillna({"flat_episodes":
        0}).reset_index()

# 9. Coverage per AHU per month: rows present
        vs rows expected, first to last
        timestamp.
q["month"] = q["timestamp_utc"].str[:7]
q["running"] = running.astype(int)
coverage = (
    q.groupby(["equipment_id", "month"])
    .agg(
        rows=("timestamp_utc", "size"),
        first_s=("epoch_s", "min"),
        last_s=("epoch_s", "max"),
        running_rows=("running", "sum"),
        dropout_rows=("dropout", "sum"),
        sp_zero_rows=("sp_zero", "sum"),
        flat_rows=("flatline", "sum"),
    )
    .reset_index()
)
coverage["expected_rows"] = (coverage["last_s"]
        - coverage["first_s"]) / STEP_S + 1
coverage["coverage_pct"] = 100.0 *
        coverage["rows"] /
        coverage["expected_rows"]
```

```
coverage = coverage.drop(columns=["first_s",
          "last_s"])

# 10. Challenge: |discharge - setpoint| > 2.78
          degC while running, confirmed for 6
          rows.
# Two variants: "ungated" trusts every row;
          "gated" skips dropouts and the missing
          (0) setpoint.
err_c = (q["dat_c"] - q["dat_sp_c"]).abs()
variants = {
    "ungated": running & (err_c > TOL_C),
    "gated": running & (q["dropout"] == 0) &
          (q["sp_zero"] == 0) & (err_c > TOL_C),
}
parts = []
for name, flag in variants.items():
    raw = flag.astype(int)
    streak_id, streak_len = streaks(raw)
    fault = ((raw == 1) & (streak_len >=
          CONFIRM_ROWS)).astype(int)
    parts.append(
        q[["equipment_id", "timestamp_utc",
          "month"]].assign(
            variant=name, raw_fault=raw,
          streak_id=streak_id, fault=fault
        )
    )
dat_rows = pd.concat(parts, ignore_index=True)
dat_rows["ungated_fault"] =
          (dat_rows["variant"] == "ungated") *
          dat_rows["fault"]
dat_rows["gated_fault"] = (dat_rows["variant"]
          == "gated") * dat_rows["fault"]
dat_month = (
    dat_rows.groupby(["equipment_id", "month"])
          [["ungated_fault", "gated_fault"]]
    .sum()
    .mul(5 / 60)
    .reset_index()
)
dat_month = dat_month.rename(
```

```
    columns={"ungated_fault":
         "ungated_fault_h", "gated_fault":
         "gated_fault_h"}
)
dat_rows = dat_rows.drop(columns=
         ["ungated_fault", "gated_fault",
         "month"])

RESULTS = {
    "quality_rows": q.drop(columns=["epoch_s",
         "month", "running"]),
    "checks": checks,
    "flat_episodes": flat_episodes,
    "coverage": coverage,
    "dat_rows": dat_rows,
    "dat_month": dat_month,
}

print("Quality checks per AHU:")
print(checks.round(2).T.to_string(header=False))
print("\nThe 7 dropout rows on AHU_1 (every
         sensor reads 0 degF = -17.78 degC):")
dropped = q[(q["equipment_id"] == "AHU_1") &
         (q["dropout"] == 1)]
print(dropped[["timestamp_utc", "dat_c",
         "dat_sp_c"]].round(2).to_string(index=False))
print("\nCoverage per AHU per month (AHU_1):")
print(coverage[coverage["equipment_id"] ==
         "AHU_1"].round(1).to_string(index=False))
print("\nChallenge: DAT rule fault hours per
         AHU per month (ungated vs gated):")
print(dat_month.round(2).to_string(index=False))
print("\nTotals:")
print(
    dat_month.groupby("equipment_id")
         [["ungated_fault_h", "gated_fault_h"]]
    .sum()
    .round(2)
    .to_string()
)
```

day15_sql.py

```
"""Day 15, SQL: data quality before you trust a
        rule, then the Unit 1 challenge (a DAT
        rule).

    uv run python lessons/course.py setup
        # once: loads the CSVs into
        var/lessons.sqlite
    uv run python lessons/unit1-
        sqlite/day15_sql.py

The same computation as day15_pandas.py, done
        by SQLite on the same rows. SQLite
        does all the work;
pandas only turns the result rows into a table
        for printing.
Compare the printout with day15_pandas.py: the
        numbers should match.
"""

import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT /
        "var" / "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv
        run python lessons/course.py setup")
conn = sqlite3.connect(DB)

PARAMS = {
    "pa_per_inh2o": 249.08891,  # the historian
        logged inches of water; the math is SI
        (pascals)
    "flat_rows": 12,  # an identical value for
        12 rows x 5 min = 1 hour, fan running,
        is suspicious
```

```
    "tol": 2.78,  # |discharge - setpoint|
        above this is a fault (5 degF is 2.78
        degC)
    "confirm": 6,  # 6 rows x 5 min = 30
        minutes
    "step_s": 300,  # the historian logs every
        5 minutes
    "zero_f_in_c": (0 - 32.0) * 5.0 / 9.0,  #
        what a dropout looks like after
        conversion: -17.777...
    "temp_lo": -40.0,  # a sensor's physical
        range
    "temp_hi": 60.0,
    "static_lo": 0.0,
    "static_hi": 1500.0,
    "speed_lo": 0.0,
    "speed_hi": 100.0,
}

# One query, one CTE per step. ahu_1 and ahu_2
        are the CSVs loaded as tables, same
        column names.
QUALITY_SQL = """
CREATE TEMP TABLE q AS
WITH h AS (                                    -
        - 1. load: both AHUs, the columns we
        need
  SELECT 'AHU_1' AS equipment_id,
        timestamp_utc, supply_fan_status,
        supply_fan_speed_pct, da_p_inwc,
        discharge_air_temp_f, dat_reset_f,
        mixed_air_temp_f, return_air_temp_f,
        outside_air_temp_f
  FROM ahu_1
  UNION ALL
  SELECT 'AHU_2', timestamp_utc,
        supply_fan_status,
        supply_fan_speed_pct, da_p_inwc,
        discharge_air_temp_f, dat_reset_f,
        mixed_air_temp_f, return_air_temp_f,
        outside_air_temp_f
  FROM ahu_2
), si AS (                                     -
        - 2. SI units (raw degF stays for the
        dropout test)
```

```
  SELECT h.*,
         (discharge_air_temp_f - 32.0) * 5.0 /
         9.0 AS dat_c,
         (dat_reset_f - 32.0) * 5.0 / 9.0
         AS dat_sp_c,
         (mixed_air_temp_f - 32.0) * 5.0 / 9.0
         AS mat_c,
         (return_air_temp_f - 32.0) * 5.0 / 9.0
         AS rat_c,
         (outside_air_temp_f - 32.0) * 5.0 /
         9.0   AS oat_c,
         da_p_inwc * :pa_per_inh2o
         AS duct_static_pa
  FROM h
), flags AS (                                   -
         - 3-5. quality flags; 6a. flat marks;
         7. time step
  SELECT *,
         CASE WHEN discharge_air_temp_f = 0 OR
         mixed_air_temp_f = 0 OR
         return_air_temp_f = 0
              THEN 1 ELSE 0 END AS dropout,
         CASE WHEN supply_fan_status = 1 AND
         dat_reset_f = 0 THEN 1 ELSE 0 END AS
         sp_zero,
         CASE WHEN dat_c NOT BETWEEN :temp_lo
         AND :temp_hi
                OR mat_c NOT BETWEEN :temp_lo
         AND :temp_hi
                OR rat_c NOT BETWEEN :temp_lo
         AND :temp_hi
                OR oat_c NOT BETWEEN :temp_lo
         AND :temp_hi
                OR duct_static_pa NOT BETWEEN
         :static_lo AND :static_hi
                OR supply_fan_speed_pct NOT
         BETWEEN :speed_lo AND :speed_hi
              THEN 1 ELSE 0 END AS
         out_of_range,
         CASE WHEN discharge_air_temp_f =
         LAG(discharge_air_temp_f) OVER w
              THEN 0 ELSE 1 END AS new_flat,
         unixepoch(timestamp_utc) AS epoch_s,
```

```
        unixepoch(timestamp_utc) -
        LAG(unixepoch(timestamp_utc)) OVER w
        AS step_s
  FROM si
  WINDOW w AS (PARTITION BY equipment_id ORDER
        BY timestamp_utc)
), flat_ids AS (                                -
        - 6b. running sum of marks = streak id
  SELECT *, SUM(new_flat) OVER (PARTITION BY
        equipment_id ORDER BY timestamp_utc)
        AS flat_id
  FROM flags
), flat AS (                                    -
        - 6c. row in streak; streak size and
        fan rows
  SELECT *,
        ROW_NUMBER() OVER (PARTITION BY
        equipment_id, flat_id ORDER BY
        timestamp_utc) AS flat_len,
        COUNT(*)               OVER (PARTITION
        BY equipment_id, flat_id) AS
        island_rows,
        SUM(supply_fan_status) OVER (PARTITION
        BY equipment_id, flat_id) AS
        island_running
  FROM flat_ids
)
SELECT equipment_id, timestamp_utc,
        supply_fan_status, dat_c, dat_sp_c,
        mat_c, rat_c, oat_c,
      duct_static_pa,
      dropout, sp_zero, out_of_range, flat_id,
        flat_len,
      CASE WHEN island_rows >= :flat_rows AND
        island_running = island_rows
           THEN 1 ELSE 0 END AS flatline,
      step_s, epoch_s, substr(timestamp_utc,
        1, 7) AS month
FROM flat
"""

FLAT_EPISODES_SQL = """
SELECT equipment_id, flat_id,
        min(timestamp_utc) AS start_utc,
        max(flat_len) AS rows,
```

```
       min(dat_c) AS value_c
FROM q
WHERE flatline = 1
GROUP BY equipment_id, flat_id
ORDER BY equipment_id, flat_id
"""

# 8. One line of checks per AHU.
CHECKS_SQL = """
SELECT equipment_id,
       sum(dropout) AS dropout_rows,
       sum(sp_zero) AS sp_zero_rows,
       sum(out_of_range) AS out_of_range_rows,
       sum(flatline) AS flatline_rows,
       sum(CASE WHEN step_s <> :step_s THEN 1
         ELSE 0 END) AS step_gaps,
       max(step_s) AS max_step_s,
       (SELECT count(DISTINCT f.flat_id) FROM q
         f
         WHERE f.equipment_id = q.equipment_id
         AND f.flatline = 1) AS flat_episodes,
       sum(CASE WHEN dat_c = -17.78 THEN 1 ELSE
         0 END) AS dat_eq_minus_17_78,
       sum(CASE WHEN abs(dat_c - :zero_f_in_c)
         < 0.01 THEN 1 ELSE 0 END)
         AS dat_within_0_01_of_zero_f
FROM q
GROUP BY equipment_id
ORDER BY equipment_id
"""

# 9. Coverage per AHU per month: rows present
        vs rows expected, first to last
        timestamp.
COVERAGE_SQL = """
SELECT equipment_id, month,
       count(*) AS rows,
       CAST(sum(supply_fan_status) AS INTEGER)
         AS running_rows,
       sum(dropout) AS dropout_rows,
       sum(sp_zero) AS sp_zero_rows,
       sum(flatline) AS flat_rows,
```

```
       (max(epoch_s) - min(epoch_s)) / :step_s
         + 1 AS expected_rows,
       100.0 * count(*) / ((max(epoch_s) -
         min(epoch_s)) / :step_s + 1) AS
         coverage_pct
FROM q
GROUP BY equipment_id, month
ORDER BY equipment_id, month
"""

# 10. Challenge: |discharge - setpoint| > 2.78
        degC while running, confirmed for 6
        rows.
# Two variants: "ungated" trusts every row;
        "gated" skips dropouts and the missing
        (0) setpoint.
DAT_SQL = """
CREATE TEMP TABLE dat_rows AS
WITH base AS (
  SELECT equipment_id, timestamp_utc, month,
         supply_fan_status, dropout, sp_zero,
         abs(dat_c - dat_sp_c) AS err_c
  FROM q
), v AS (
  SELECT 'ungated' AS variant, equipment_id,
         timestamp_utc, month,
         CASE WHEN supply_fan_status = 1 AND
         err_c > :tol THEN 1 ELSE 0 END AS
         raw_fault
  FROM base
  UNION ALL
  SELECT 'gated', equipment_id, timestamp_utc,
         month,
         CASE WHEN supply_fan_status = 1 AND
         dropout = 0 AND sp_zero = 0 AND err_c
         > :tol
              THEN 1 ELSE 0 END
  FROM base
), marks AS (
  SELECT *, CASE WHEN raw_fault =
         LAG(raw_fault) OVER w THEN 0 ELSE 1
         END AS new_streak
  FROM v
```

```
  WINDOW w AS (PARTITION BY variant,
         equipment_id ORDER BY timestamp_utc)
), ids AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY
         variant, equipment_id ORDER BY
         timestamp_utc)
           AS streak_id
  FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY
         variant, equipment_id, streak_id ORDER
         BY timestamp_utc)
           AS streak_len
  FROM ids
)
SELECT variant, equipment_id, timestamp_utc,
         month, raw_fault, streak_id,
       CASE WHEN raw_fault = 1 AND streak_len
         >= :confirm THEN 1 ELSE 0 END AS fault
FROM ranked
"""
DAT_MONTH_SQL = """
SELECT equipment_id, month,
       sum(CASE WHEN variant = 'ungated' AND
         fault = 1 THEN 1 ELSE 0 END) * 5 /
         60.0
         AS ungated_fault_h,
       sum(CASE WHEN variant = 'gated' AND
         fault = 1 THEN 1 ELSE 0 END) * 5 /
         60.0
         AS gated_fault_h
FROM dat_rows
GROUP BY equipment_id, month
ORDER BY equipment_id, month
"""

conn.execute(QUALITY_SQL, PARAMS)
conn.execute(DAT_SQL, PARAMS)

quality = pd.read_sql_query(
    "SELECT equipment_id, timestamp_utc,
         supply_fan_status, dat_c, dat_sp_c,
         mat_c, rat_c, oat_c,"
```

```
    " duct_static_pa, dropout, sp_zero,
        out_of_range, flat_len, flatline,
        step_s"
    " FROM q ORDER BY equipment_id,
        timestamp_utc",
    conn,
)
flat_episodes =
        pd.read_sql_query(FLAT_EPISODES_SQL,
        conn)
checks = pd.read_sql_query(CHECKS_SQL, conn,
        params=PARAMS)
coverage = pd.read_sql_query(COVERAGE_SQL,
        conn, params=PARAMS)
dat_rows = pd.read_sql_query(
    "SELECT equipment_id, timestamp_utc,
        variant, raw_fault, streak_id, fault"
    " FROM dat_rows ORDER BY variant,
        equipment_id, timestamp_utc",
    conn,
)
dat_month = pd.read_sql_query(DAT_MONTH_SQL,
        conn)

RESULTS = {
    "quality_rows": quality,
    "checks": checks,
    "flat_episodes": flat_episodes,
    "coverage": coverage,
    "dat_rows": dat_rows,
    "dat_month": dat_month,
}

print("Quality checks per AHU:")
print(checks.round(2).T.to_string(header=False))
print("\nThe 7 dropout rows on AHU_1 (every
        sensor reads 0 degF = -17.78 degC):")
dropped = quality[(quality["equipment_id"] ==
        "AHU_1") & (quality["dropout"] == 1)]
print(dropped[["timestamp_utc", "dat_c",
        "dat_sp_c"]].round(2).to_string(index=False))
print("\nCoverage per AHU per month (AHU_1):")
print(coverage[coverage["equipment_id"] ==
        "AHU_1"].round(1).to_string(index=False))
```

```
print("\nChallenge: DAT rule fault hours per
          AHU per month (ungated vs gated):")
print(dat_month.round(2).to_string(index=False))
print("\nTotals:")
print(
    dat_month.groupby("equipment_id")
         [["ungated_fault_h", "gated_fault_h"]]
    .sum()
    .round(2)
    .to_string()
)
```

Reading — Week 3

The lessons first, then these for exact behavior.

- SQLite window functions
- pandas: windowing operations
- SQLite query planner
- SQLite EXPLAIN QUERY PLAN
- Open-FDD SQL cookbook
- TimescaleDB hypertables
- TimescaleDB continuous aggregates